lasso_psm_analysis

nakada shunichi

1. データ読み込み

```
# ==============================================================================
# 0. 初期設定・パッケージ読み込み
# ==============================================================================
library(tinytex)
library(here)
```

here() starts at /Users/snakada/Library/CloudStorage/GoogleDrive-snakada@g.ecc.u-tokyo.ac.jp/マイドライブ/Peru_work

```
library(haven)
library(glmnet)
```

Loading required package: Matrix

Loaded glmnet 4.1-10

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union

```r
library(fastDummies)
library(nnet)      # 多項ロジスティック回帰
library(survey)    # 重み付き統計解析
```

```
Loading required package: grid

Loading required package: survival


Attaching package: 'survey'

The following object is masked from 'package:graphics':

    dotchart
```

```r
library(ggplot2)
library(did)       # Staggered DID用（必要に応じて）

# オブジェクトクリア
rm(list = ls(all = TRUE))

# 自作関数読み込み
source(here("myTools.R"))
```

```

Attaching package: 'psych'

The following object is masked from 'package:did':

    sim

The following objects are masked from 'package:ggplot2':

    %+%, alpha
```

```r
# ==============================================================================
# 1. データ読み込み
# ==============================================================================
path_data_file <- normalizePath(
  here("..", "output", "alldata", "merged_all_years.dta")
)
df_org <- read_dta(path_data_file)
```

2. LASSO用データ前処理（介入前データのみ）

```r
# ==============================================================================
# 2. LASSO用データ前処理（介入前データのみ）
# ==============================================================================

# 除外変数定義
drop_cols <- c(
  "caseid", "cluster_id", "strata_id", "state_id", "state",
  "treated_status", "time_period", "group_summary", "nt_ch_whz", "age",
  "v001", "v002", "v003", "v005", "v008", "v013", "v022", "v024", "v025", "v026",
  "b3", "b9", "bidx", "age_months", "b4", "v106", "v190", "hv024", "hv103",
  "hv270", "hc1", "hc27", "sampling_weight", "treated_ever", "treated",
  "treatment_cohort", "post_treatment", "dm_refrigerator", "nt_bbyfood",
  "rc_tobc_other", "ph_wtr_trt_solar", "ms_sex_never", "ms_mar_union",
  "ms_mar_never", "nt_ch_micro_vaf", "nt_solids", "nt_ch_sev_stunt",
  "nt_ch_stunt","nt_ch_mean_haz", "nt_ch_haz","nt_ch_sev_wast", "nt_ch_wast",
  "nt_ch_ovwt_ht", "nt_ch_mean_whz", "nt_ch_whz", "nt_ch_sev_underwt",
  "nt_ch_underwt", "nt_ch_ovwt_age"
)

# ID・管理用変数
grouping_cols <- c("caseid", "cluster_id", "strata_id", "state_id", "state",
                   "treated_status", "time_period", "group_summary",
                   "treatment_start", "treated_ever", "treated",
                   "treatment_cohort", "post_treatment", "sampling_weight")

# パターンマッチによる変数除外
drop_cols2 <- grep("^ch_pneumo|^ch_rotav", names(df_org), value = TRUE)
all_drop_cols <- c(drop_cols, drop_cols2, "treatment_start")

# 1. 介入前データで絞りこみ
```

```
df_pre <- df_org[df_org$treated_status %in% c("never_treated", "not_yet_treated"), ]

# 2. 年ダミー変数作成
df_pre <- dummy_cols(df_pre, select_columns = "year")

# 3. 介入群グループ化
df_pre <- df_pre %>%
  mutate(
    treatment_group = case_when(
      treatment_start == "2009" ~ 1,  # 早期介入
      treatment_start == "2011" ~ 2,  # 後期介入
      TRUE ~ 0                        # コントロール
    ),
    treatment_group = as.factor(treatment_group)
  )

cat("介入前データの群別サンプル数:\n")
```

介入前データの群別サンプル数:

```
print(table(df_pre$treatment_group))
```

```

   0    1    2 
5156  292  526 
```

```
# 4. NAの多い列を除去（10%以上NA）
initial_cols <- ncol(df_pre)
keep_cols <- names(df_pre)[sapply(df_pre, function(x) sum(is.na(x)) <= nrow(df_pre) * 0.1)]
df_pre <- df_pre[, keep_cols]
cat("NA値の多い列を", initial_cols - ncol(df_pre), "個削除しました\n")
```

NA値の多い列を 100 個削除しました

```
# 5. 共変量名の再定義（NA除去後）
all_drop_cols <- intersect(all_drop_cols, names(df_pre))
grouping_cols <- intersect(grouping_cols, names(df_pre))
covariate_names <- setdiff(names(df_pre), c(all_drop_cols, grouping_cols, "treatment_group"))

# 6. 共変量のNAがある行を除去
```

```
complete_idx <- complete.cases(df_pre[, covariate_names])
initial_rows <- nrow(df_pre)
df_pre <- df_pre[complete_idx, ]
cat("NA値を含む行を", initial_rows - nrow(df_pre), "行削除しました\n")
```

NA値を含む行を 783 行削除しました

```
cat("最終的な介入前データサイズ:", nrow(df_pre), "行 ×", ncol(df_pre), "列\n\n")
```

最終的な介入前データサイズ: 5191 行 × 245 列

3. LASSO変数選択（介入前データ）

```
# ============================================================================
# 3. LASSO変数選択（介入前データ）
# ============================================================================

# 説明変数マトリックス作成
X_matrix <- model.matrix(~ . - 1, data = df_pre[, covariate_names, drop=FALSE])
X_scaled <- scale(X_matrix)
Y <- df_pre$treatment_group

cat("説明変数の数:", ncol(X_scaled), "\n")
```

説明変数の数: 175

```
cat("目的変数の水準:", levels(Y), "\n\n")
```

目的変数の水準: 0 1 2

```
# 多項分類LASSO実行
cat("多項分類LASSO回帰を実行中...\n")
```

多項分類LASSO回帰を実行中...

```r
set.seed(123) # 再現性のため
cv_model <- cv.glmnet(X_scaled, Y, alpha = 1, family = "multinomial", nfolds = 10)

cat("最適λ (lambda.min):", cv_model$lambda.min, "\n")
```

最適λ (lambda.min): 0.0006454161

```r
cat("1se λ (lambda.1se):", cv_model$lambda.1se, "\n\n")
```

1se λ (lambda.1se): 0.003138401

```r
# 選択された変数の抽出
coef_min <- coef(cv_model, s = "lambda.min")
selected_vars <- c()
for(i in 1:length(coef_min)) {
  coef_matrix <- as.matrix(coef_min[[i]])
  class_vars <- rownames(coef_matrix)[coef_matrix[,1] != 0 & rownames(coef_matrix) != "(Intercept)"]
  selected_vars <- union(selected_vars, class_vars)
}

cat("LASSO選択変数数:", length(selected_vars), "\n")
```

LASSO選択変数数: 153

```r
if(length(selected_vars) > 0) {
  cat("選択された変数:\n")
  for(i in 1:length(selected_vars)) {
    cat(i, ":", selected_vars[i], "\n")
  }
} else {
  stop("変数が選択されませんでした。λの値を調整してください。")
}
```

選択された変数:
1 : rh_del_pv
2 : year2012
3 : ph_sani_improve
4 : ph_sani_basic
5 : ph_wtr_trt_chlor

6 : ph_wtr_trt_filt
7 : ph_wtr_trt_other
8 : ph_wtr_source
9 : ph_wtr_basic
10 : ms_afm_15
11 : ms_afm_18
12 : ms_afm_20
13 : ms_afm_22
14 : ms_age
15 : ms_afs_18
16 : ms_afs_22
17 : rc_edu
18 : rc_media_radio
19 : rc_empl
20 : rc_tobc_cig
21 : rc_tobc_smk_any
22 : rh_anc_pv
23 : rh_anc_pvskill
24 : rh_anc_4vs
25 : rh_anc_median
26 : rh_anc_parast
27 : rh_anc_prgcomp
28 : rh_prob_permit
29 : rh_prob_dist
30 : rh_prob_alone
31 : rh_prob_minone
32 : ch_ari
33 : ch_fever
34 : age_month
35 : nt_ch_micro_dwm
36 : nt_bf_start_1hr
37 : nt_bf_prelac_noBirthRecord
38 : nt_bf_status
39 : nt_liquids
40 : nt_grains
41 : nt_root
42 : nt_eggs
43 : nt_dairy
44 : nt_fed_milk
45 : nt_ch_mean_waz
46 : nt_ch_any_anem
47 : nt_ch_mod_anem
48 : nt_ch_sev_anem

49 : ch_bcg_moth
50 : ch_bcg_card
51 : ch_pent3_either
52 : ch_pent1_moth
53 : ch_pent1_card
54 : ch_pent3_card
55 : ch_polio1_either
56 : ch_polio2_either
57 : ch_polio0_moth
58 : ch_polio2_moth
59 : ch_polio0_card
60 : ch_meas_moth
61 : ch_allvac_either
62 : ch_allvac_card
63 : ch_novac_card
64 : dm_children_under12
65 : dm_cooking_fuel_traditional
66 : dm_decide_none
67 : dm_dvjustify_argue
68 : dm_dvjustify_refusesex
69 : dm_dvjustify_onereas
70 : dm_town
71 : dm_city
72 : dm_capital
73 : dm_poorest
74 : dm_middle
75 : dm_richer
76 : dm_weath_index
77 : dm_ag_land_ha
78 : dm_cooking_outdoor
79 : dm_cattle_own
80 : dm_goat_own
81 : dm_poultry_own
82 : rh_del_pltype
83 : year2005
84 : year2008
85 : year2009
86 : ph_sani_type
87 : ph_wtr_trt_boil
88 : ph_wtr_trt_stand
89 : ph_wtr_trt_appr
90 : ph_wtr_time
91 : ms_mar_stat

92 : ms_afm_25
93 : ms_afs_15
94 : ms_afs_20
95 : ms_afs_25
96 : ms_sex_recent
97 : rc_litr
98 : rc_media_tv
99 : rc_media_allthree
100 : rc_media_none
101 : rc_hins_other
102 : rh_anc_numvs
103 : rh_anc_moprg
104 : rh_anc_4mo
105 : rh_anc_iron
106 : rh_anc_bldpres
107 : rh_anc_urine
108 : rh_anc_bldsamp
109 : rh_anc_toxinj
110 : rh_prob_money
111 : ch_diar
112 : nt_bf_start_1day
113 : nt_bottle
114 : nt_milk
115 : nt_vita
116 : nt_frtveg
117 : nt_nuts
118 : ch_pent2_moth
119 : ch_meas_card
120 : ch_allvac_moth
121 : ch_novac_moth
122 : ch_stool_safe
123 : dm_cooking_fuel
124 : dm_dvjustify_burn
125 : dm_dvjustify_goout
126 : dm_justify_refusesex
127 : dm_richest
128 : dm_cooking_separate
129 : dm_sheep_own
130 : dm_age_of_HH_head
131 : year_2005
132 : rh_del_ces
133 : year2010
134 : ph_wtr_trt_none

135 : ph_wtr_improve
136 : rc_litr_cats
137 : rc_media_newsp
138 : rc_hins_any
139 : rh_anc_neotet
140 : ch_size_birth
141 : nt_ageapp_bf
142 : nt_formula
143 : nt_mmf
144 : ch_polio3_either
145 : ch_polio3_card
146 : ch_meas_either
147 : ch_card_ever_had
148 : dm_dvjustify_neglect
149 : dm_rural
150 : dm_cooking_house
151 : dm_female_headed
152 : year_2009
153 : year_2010

4. 多項GLM-PSM（介入前データで学習）

```
# ============================================================================
# 4. 多項GLM-PSM（介入前データで学習）
# ============================================================================
if(length(selected_vars) > 0) {
  # LASSO と同じ model.matrix 処理を適用
  psm_matrix <- model.matrix(~ . - 1, data = df_pre[, covariate_names, drop=FALSE])

  # 選択された変数のみ抽出
  psm_covariates <- as.data.frame(psm_matrix[, selected_vars, drop=FALSE])

  # 目的変数と管理変数を追加
  psm_data <- cbind(
    treatment_group = df_pre$treatment_group,
    psm_covariates,
    df_pre[, intersect(grouping_cols, names(df_pre))]
  )

  # 多項ロジスティック回帰で傾向スコア推定
  psm_formula <- as.formula(paste("treatment_group ~", paste(selected_vars, collapse = " + ")))
```

```r
cat("\n多項ロジスティック回帰で傾向スコア推定中...\n")
psm_model <- multinom(psm_formula, data = psm_data, trace = FALSE)

# 傾向スコア（各群の確率）を計算
pscore_matrix <- predict(psm_model, type = "probs")

# デバッグ情報
cat("pscore_matrix の次元:", dim(pscore_matrix), "\n")
cat("treatment_group の水準:", levels(psm_data$treatment_group), "\n")
cat("treatment_group の値:", table(psm_data$treatment_group), "\n")

# データフレーム形式に変換
if(is.vector(pscore_matrix)) {
  # 2群の場合の処理
  pscore_df <- data.frame(
    group_0 = 1 - pscore_matrix,
    group_1 = pscore_matrix
  )
  names(pscore_df) <- paste0("prob_", levels(psm_data$treatment_group))
} else {
  # 3群以上の場合
  pscore_df <- as.data.frame(pscore_matrix)
  # 列名を明示的に設定
  names(pscore_df) <- paste0("prob_", levels(psm_data$treatment_group))
}

# IPTW重み計算（より安全な方法）
psm_data$iptw_weight <- numeric(nrow(psm_data))

for(i in 1:nrow(psm_data)) {
  group_level <- as.character(psm_data$treatment_group[i])
  prob_col <- paste0("prob_", group_level)

  if(prob_col %in% names(pscore_df)) {
    psm_data$iptw_weight[i] <- 1 / pscore_df[i, prob_col]
  } else {
    cat("警告: 群", group_level, "の確率列が見つかりません\n")
    psm_data$iptw_weight[i] <- 1  # デフォルト重み
  }
}

cat("PSM完了。重みの要約統計:\n")
```

```r
  print(summary(psm_data$iptw_weight))

  # 極端な重みの確認
  extreme_weights <- psm_data$iptw_weight > quantile(psm_data$iptw_weight, 0.99, na.rm = TRUE)
  if(sum(extreme_weights, na.rm = TRUE) > 0) {
    cat("警告: 極端に大きな重みが", sum(extreme_weights, na.rm = TRUE), "個あります\n")
  }

} else {
  stop("選択された変数がありません。")
}
```

```
多項ロジスティック回帰で傾向スコア推定中...
pscore_matrix の次元: 5191 3
treatment_group の水準: 0 1 2
treatment_group の値: 4450 264 477
PSM完了。重みの要約統計:
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.000   1.001   1.041   1.828   1.238 104.891
警告: 極端に大きな重みが 52 個あります
```

5. 介入後＋コントロールデータへの適用

```r
# =============================================================================
# 5. 介入後＋コントロールデータへの適用
# =============================================================================

# 介入後＋コントロールデータ抽出
df_post <- df_org[df_org$treated_status %in% c("never_treated", "early_treated", "late_treated"), ]

# 介入群の再定義
df_post <- df_post %>%
  mutate(
    treatment_group = case_when(
      treated_status == "never_treated" ~ 0,
      treated_status == "early_treated" ~ 1,
      treated_status == "late_treated" ~ 2,
      TRUE ~ 0
    ),
```

```
  treatment_group = as.factor(treatment_group)
 )

cat("\n介入後データの群別サンプル数:\n")
```

介入後データの群別サンプル数:

```
print(table(df_post$treatment_group))
```

```
   0    1    2 
5156  466  260 
```

```
# 年ダミー変数作成
df_post <- dummy_cols(df_post, select_columns = "year")

# 共変量を介入前と同じ形式に調整
missing_vars <- setdiff(selected_vars, names(df_post))
if(length(missing_vars) > 0) {
 cat("警告: 以下の変数が介入後データに存在しません:", paste(missing_vars, collapse = ", "), "\n")
 # 欠損変数を0で補完
 for(var in missing_vars) {
  df_post[[var]] <- 0
 }
}
```

警告: 以下の変数が介入後データに存在しません: year2012, year2005, year2008, year2009, year2010

```
# NAを含む行を除去
post_complete_idx <- complete.cases(df_post[, c(selected_vars, grouping_cols, "nt_ch_stunt")])
df_post <- df_post[post_complete_idx, ]

# 学習済みモデルで傾向スコア予測
cat("学習済みモデルで介入後データに傾向スコア適用中...\n")
```

学習済みモデルで介入後データに傾向スコア適用中...

```r
# 介入後データも model.matrix で処理（PSM学習時と同じ形式に）
post_matrix <- model.matrix(~ . - 1, data = df_post[, covariate_names, drop=FALSE])
post_covariates <- as.data.frame(post_matrix[, selected_vars, drop=FALSE])

# 予測用データフレーム作成
predict_data <- cbind(
  treatment_group = df_post$treatment_group,
  post_covariates
)

# 傾向スコア予測
post_pscore_matrix <- predict(psm_model, newdata = predict_data, type = "probs")

# デバッグ情報
cat("post_pscore_matrix の次元:", dim(post_pscore_matrix), "\n")
```

post_pscore_matrix の次元: 5069 3

```r
cat("df_post の treatment_group:", table(df_post$treatment_group), "\n")
```

df_post の treatment_group: 4450 404 215

```r
# データフレーム形式に変換
if(is.vector(post_pscore_matrix)) {
  # 2群の場合
  post_pscore_df <- data.frame(
    group_0 = 1 - post_pscore_matrix,
    group_1 = post_pscore_matrix
  )
  names(post_pscore_df) <- paste0("prob_", levels(df_post$treatment_group))
} else {
  # 3群以上の場合
  post_pscore_df <- as.data.frame(post_pscore_matrix)
  names(post_pscore_df) <- paste0("prob_", levels(df_post$treatment_group))
}

# IPTW重み計算（安全な方法）
df_post$iptw_weight <- numeric(nrow(df_post))

for(i in 1:nrow(df_post)) {
```

```r
  group_level <- as.character(df_post$treatment_group[i])
  prob_col <- paste0("prob_", group_level)

  if(prob_col %in% names(post_pscore_df)) {
    prob_value <- post_pscore_df[i, prob_col]
    # 極小確率値の処理（ゼロ除算防止）
    if(prob_value > 0.001) {
      df_post$iptw_weight[i] <- 1 / prob_value
    } else {
      df_post$iptw_weight[i] <- 1 / 0.001  # 最大重み制限
      if(i <= 10) cat("警告: 極小確率値を調整しました（行", i, ")\n")
    }
  } else {
    cat("警告: 群", group_level, "の確率列が見つかりません（行", i, ")\n")
    df_post$iptw_weight[i] <- 1
  }
}

cat("介入後データの重み計算完了\n")
```

介入後データの重み計算完了

```r
print(summary(df_post$iptw_weight))
```

```
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max. 
   1.000    1.001    1.039   81.839    1.245 1000.000 
```

```r
# サンプリングウェイトとの組み合わせ（v005を1,000,000で割って正規化）
df_post$sampling_weight_norm <- df_post$v005 / 1000000
df_post$w_combined <- df_post$iptw_weight * df_post$sampling_weight_norm

# 極端な重みを調整（オプション）
weight_quantiles <- quantile(df_post$w_combined, c(0.01, 0.99), na.rm = TRUE)
df_post$w_combined_trimmed <- pmax(pmin(df_post$w_combined, weight_quantiles[2]), weight_quantiles[1])

cat("重み付きデータ準備完了。最終サンプル数:", nrow(df_post), "\n")
```

重み付きデータ準備完了。最終サンプル数: 5069

```r
cat("重みの要約統計:\n")
```

重みの要約統計:

```r
print(summary(df_post$w_combined_trimmed))
```

```
    Min.   1st Qu.    Median      Mean   3rd Qu.      Max. 
 0.07067   0.37963   0.97888  48.87621   1.87642 891.45100 
```

6. 重み付き多群比較分析

```r
# ==============================================================================
# 6. 重み付き多群比較分析
# ==============================================================================

# Survey design作成
design_post <- svydesign(
  ids = ~v001,     # DHS Primary Sampling Unit
  strata = ~v022,  # DHS strata
  weights = ~w_combined_trimmed,
  data = df_post,
  nest = TRUE
)

# 孤立PSUの処理
options(survey.lonely.psu = "adjust")

cat("\n=== 重み付き群別平均値 ===\n")
```

```
=== 重み付き群別平均値 ===
```

```r
group_means <- svyby(~nt_ch_stunt, ~treatment_group, design = design_post, svymean, na.rm = TRUE)
print(group_means)
```

```
  treatment_group nt_ch_stunt         se
0               0   0.2389067 0.01102037
1               1   0.4510059 0.03172132
2               2   0.3288206 0.03803110
```

```r
# 2群間t検定
cat("\n=== 2群間t検定 ===\n")
```

```
=== 2群間t検定 ===
```

```r
# コントロール vs 早期介入
cat("コントロール群 vs 早期介入群:\n")
```

```
コントロール群 vs 早期介入群:
```

```r
design_01 <- subset(design_post, treatment_group %in% c(0, 1))
design_01 <- update(design_01, treatment_group = droplevels(treatment_group))
if(nrow(design_01$variables) > 0 && length(unique(design_01$variables$treatment_group)) == 2) {
  result_01 <- svyttest(nt_ch_stunt ~ treatment_group, design = design_01)
  print(result_01)
} else {
  cat("十分なサンプルがありません\n")
}
```

```
    Design-based t-test

data:  nt_ch_stunt ~ treatment_group
t = 6.316, df = 2322, p-value = 3.206e-10
alternative hypothesis: true difference in mean is not equal to 0
95 percent confidence interval:
 0.1462471 0.2779513
sample estimates:
difference in mean
         0.2120992
```

```r
# コントロール vs 後期介入
cat("\nコントロール群 vs 後期介入群:\n")
```

```
コントロール群 vs 後期介入群:
```

```r
design_02 <- subset(design_post, treatment_group %in% c(0, 2))
design_02 <- update(design_02, treatment_group = droplevels(treatment_group))
if(nrow(design_02$variables) > 0 && length(unique(design_02$variables$treatment_group)) == 2) {
  result_02 <- svyttest(nt_ch_stunt ~ treatment_group, design = design_02)
  print(result_02)
} else {
  cat("十分なサンプルがありません\n")
}
```

```
    Design-based t-test

data:  nt_ch_stunt ~ treatment_group
t = 2.2708, df = 2240, p-value = 0.02325
alternative hypothesis: true difference in mean is not equal to 0
95 percent confidence interval:
 0.01226593 0.16756183
sample estimates:
difference in mean
        0.08991388
```

```r
# 早期介入 vs 後期介入
cat("\n早期介入群 vs 後期介入群:\n")
```

```
早期介入群 vs 後期介入群:
```

```r
design_12 <- subset(design_post, treatment_group %in% c(1, 2))
design_12 <- update(design_12, treatment_group = droplevels(treatment_group))
if(nrow(design_12$variables) > 0 && length(unique(design_12$variables$treatment_group)) == 2) {
  result_12 <- svyttest(nt_ch_stunt ~ treatment_group, design = design_12)
  print(result_12)
} else {
  cat("十分なサンプルがありません\n")
}
```

```
    Design-based t-test

data:  nt_ch_stunt ~ treatment_group
```

```
t = -2.4672, df = 359, p-value = 0.01408
alternative hypothesis: true difference in mean is not equal to 0
95 percent confidence interval:
 -0.21957853 -0.02479212
sample estimates:
difference in mean
       -0.1221853
```

```
# 3群ANOVA
cat("\n=== 3群ANOVA ===\n")
```

```
=== 3群ANOVA ===
```

```
anova_model <- svyglm(nt_ch_stunt ~ factor(treatment_group), design = design_post)
print(summary(anova_model))
```

```
Call:
svyglm(formula = nt_ch_stunt ~ factor(treatment_group), design = design_post)

Survey design:
svydesign(ids = ~v001, strata = ~v022, weights = ~w_combined_trimmed,
    data = df_post, nest = TRUE)

Coefficients:
                        Estimate Std. Error t value Pr(>|t|)
(Intercept)              0.23891    0.01102  21.679  < 2e-16 ***
factor(treatment_group)1  0.21210    0.03358   6.316 3.17e-10 ***
factor(treatment_group)2  0.08991    0.03960   2.271   0.0232 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 0.2370007)

Number of Fisher Scoring iterations: 2
```

```
print(anova(anova_model))
```

```
NULL
```

7. Staggered DID（オプション）

```
# ==============================================================================
# 7. Staggered DID（オプション）
# ==============================================================================

cat("\n=== Staggered DID分析 ===\n")
```

```
=== Staggered DID分析 ===
```

```
# DID用データ準備
did_data <- df_org %>%
  mutate(
    G = case_when(
      treated_status == "never_treated" ~ 0,
      treatment_start == "2009" ~ 2009,
      treatment_start == "2011" ~ 2011,
      TRUE ~ 0
    ),
    T = as.integer(year),
    D = as.integer(treated_status != "never_treated" & post_treatment == 1),
    unit_id = as.integer(as.factor(paste(caseid, cluster_id, sep = "_")))
  ) %>%
  filter(!is.na(nt_ch_stunt), !is.na(G), !is.na(T)) %>%
  arrange(unit_id, T)

# IPTWウェイトをDIDデータにマージ（caseid等で結合）
if("w_combined_trimmed" %in% names(df_post)) {
  did_weights <- df_post %>%
    select(caseid, year, w_combined_trimmed) %>%
    mutate(T = as.integer(year))

  did_data <- did_data %>%
    left_join(did_weights, by = c("caseid", "T")) %>%
    mutate(weights = ifelse(is.na(w_combined_trimmed), 1, w_combined_trimmed))
} else {
  did_data$weights <- 1
}

# DID実行（did パッケージ使用）
```

```r
if(requireNamespace("did", quietly = TRUE)) {
  tryCatch({
    did_result <- did::att_gt(
      yname = "nt_ch_stunt",
      tname = "T",
      idname = "unit_id",
      gname = "G",
      data = did_data,
      weightsname = "weights",
      panel = TRUE,
      control_group = "nevertreated"
    )

    # 結果要約
    did_summary <- did::aggte(did_result, type = "group")
    print(did_summary)

    # 動的効果のプロット
    did_dynamic <- did::aggte(did_result, type = "dynamic")
    print(did_dynamic)

  }, error = function(e) {
    cat("DID分析でエラーが発生しました:", e$message, "\n")
    cat("データ構造やサンプル数を確認してください\n")
  })
} else {
  cat("didパッケージがインストールされていません。install.packages('did')でインストールしてください\n")
}
```

```
Warning in pre_process_did(yname = yname, tname = tname, idname = idname, :
Dropped 6700 observations while converting to balanced panel.
```

```
DID分析でエラーが発生しました: All observations dropped to converted data to balanced panel. Consider setting `panel =
データ構造やサンプル数を確認してください
```

8. 結果の保存・可視化

==

8. 結果の保存・可視化

==

writeLines(c(“=== 分析完了 ===”, “主要結果:”, paste(“1. LASSO選択変数数:”, length(selected_vars)), paste(“2. 最終分析サンプル数:”, nrow(df_post)), “3. 群別サンプル数:”)) print(table(df_post$treatment_group))

重みの分布可視化

ggplot(df_post, aes(x = w_combined_trimmed, fill = treatment_group)) + geom_histogram(alpha = 0.6, bins = 50, position = “identity”) + scale_x_log10() + labs(x = “Combined Weight (log scale)”, y = “Frequency”, fill = “Treatment Group”, title = “Distribution of Combined IPTW × Sampling Weights”) + theme_minimal()

群別アウトカムの可視化

outcome_plot <- ggplot(df_post, aes(x = factor(treatment_group), y = nt_ch_stunt, weight = w_combined_trimmed)) + geom_boxplot(aes(fill = factor(treatment_group)), alpha = 0.7) + labs(x = “Treatment Group (0=Control, 1=Early, 2=Late)”, y = “Stunting Outcome”, fill = “Group”, title = “Weighted Distribution of Stunting by Treatment Group”) + theme_minimal()

print(outcome_plot)